

Initiation à Git

Table des matières

1. Nathaniel Vaur Henel
2. Introduction à Git
3. Concepts fondamentaux
4. Fonctionnement simplifié de Git
5. Premiers pas avec Git
6. Travailler avec un dépôt distant
7. Branches
8. Cas pratique guidé
9. Erreurs courantes et corrections
10. Bonnes pratiques
11. Glossaire
12. Quiz final
13. Conclusion du cours

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**



Nathaniel Vaur Henel

L'auteur

Auteur

👨 Nathaniel Vaur Henel 🤘

- 👨‍🏫 Formateur
- 👨‍💻 Tech leader
- 🧉 Expert Java
- 💻 20 ans d'expérience en développement, principalement en Java
- 📜 Créateur de contenus pour dévs : dev-sensei.digicrafters.fr
- Discord : `@nathanielvaurhenel`



framagit.org/nathvh/me



dev-sensei.digicrafters.fr



linkedin.com/in/nathaniel-vaur-henel



Introduction à Git

L’intérêt de Git

Git, c’est quoi exactement ? À quoi ça sert, et pourquoi tout le monde l’utilise ? ✋

Git est un système de gestion de versions qui permet :

- De suivre les modifications apportées à un projet 📜
- De collaborer efficacement à plusieurs 🤝
- De revenir facilement à un état antérieur si besoin BACK

> 🧔🏽 J’utilise Git pour sauvegarder mon travail en tout genre. Il me permet de travailler sur tous mes projets, de développement ou autre depuis n’importe quelle machine. Par exemple, tous mes cours sont versionnés et accessibles depuis n’importe où. C’est un outil indispensable pour moi.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Plus qu'une sauvegarde

Est-ce que Git est juste une sauvegarde améliorée ? ✋

Git est un **outil de gestion de versions distribué** :

- Chaque utilisateur possède une copie complète de l'historique du projet 💾
- Permet un **travail hors-ligne**
- Utilisé dans des millions de projets, du simple document texte à de grands logiciels open-source 💻

La conception de Git

Quelles problématiques a-t-il résolues ? ✋

- Git a été développé en 2005 par **Linus Torvalds** pour gérer le code du noyau Linux 🐧
- Objectifs :
  - Un système **rapide**, **fiable** et **distribué**
  - Permettre un travail en parallèle sans conflits majeurs
  - Une **sécurité** forte sur l'historique des modifications 🔐

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Problèmes que Git résout

Qu'est-ce que Git rend plus simple dans un projet collaboratif ? ✋

Grâce à Git, on peut :

- **Versionner** tous les fichiers d'un projet 📁
- **Collaborer** à plusieurs sans écraser le travail des autres 👥
- **Explorer l'historique** pour retrouver la source d'un bug 🐛
- **Créer des branches** pour tester une idée sans tout casser 🌱

Ce que Git n’est pas

Est-ce que Git remplace Dropbox ou Google Drive ? ✋

❌ Non, Git **n’est pas un hébergeur de fichiers** :

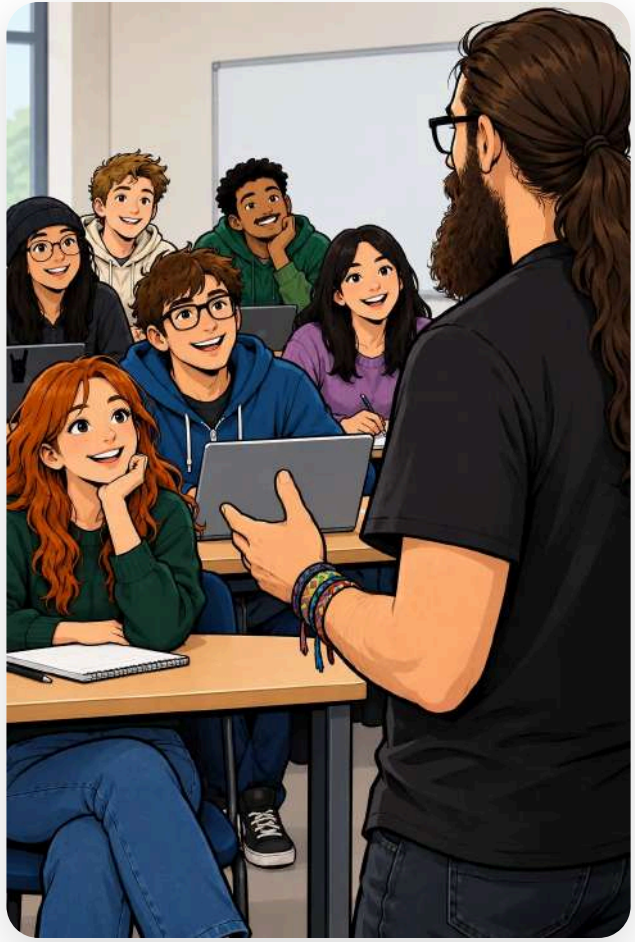
- Il ne synchronise pas automatiquement des fichiers comme Dropbox ☁️
- Ce **n’est pas GitHub** non plus :
  - Git est un outil local
  - GitHub est une **plateforme d’hébergement Git à distance** 🌐

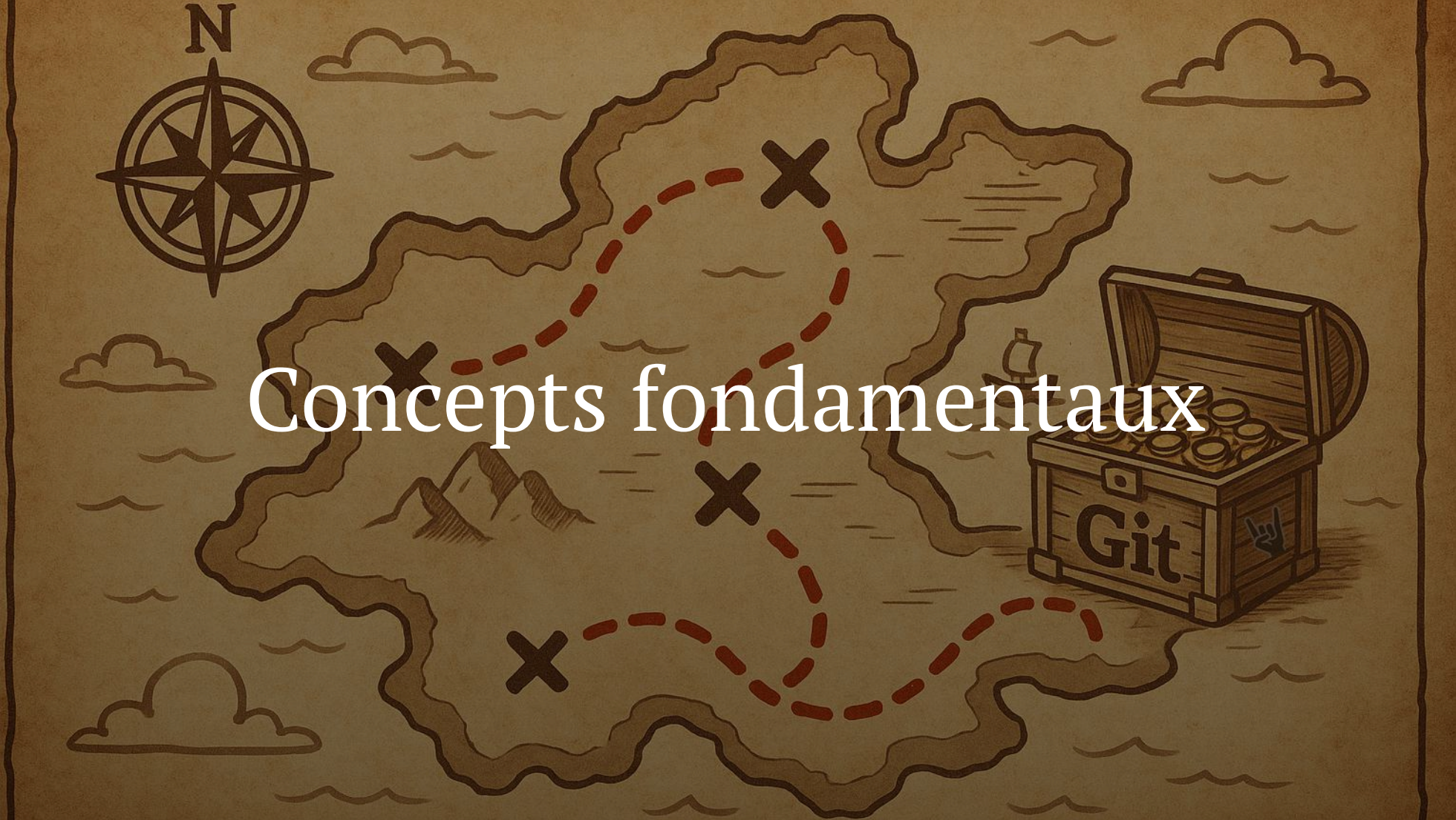
> 🧔 Par contre, tu peux sauvegarder ton dépôt Git sur GitHub ou GitLab pour maitriser ton historique et collaborer avec d’autres personnes.

Questions
Introduction à Git

Vos questions ✋
Vos demandes d'ajout ✋

- ?





Concepts fondamentaux

Comprendre les concepts de base de Git

Avant d'utiliser Git, qu'est-ce qu'il faut absolument comprendre ? ✋

Pour bien utiliser Git, il est essentiel de :

- Comprendre ce que veut dire versionner, committer, brancher
- Se familiariser avec le vocabulaire Git
- Distinguer l'outil Git des plateformes comme GitHub

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Versionner, committer, brancher : définitions simples

Ces termes semblent techniques, mais à quoi correspondent-ils vraiment ? ✋

- **Versionner** : garder un historique des modifications apportées à un fichier 📚
- **Committer** : enregistrer un instantané des changements réalisés 📸
- **Brancher** : créer une copie parallèle du projet pour tester ou développer sans toucher au reste 🌿

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Vocabulaire de base

Quels sont les mots-clés à connaître absolument ? ✋

- **Dépôt (repository)** : dossier contenant les fichiers versionnés et l'historique Git 📁
- **Commit** : enregistrement d'un ensemble de modifications ✅
- **Branche** : ligne de développement indépendante de la branche principale 🌳
- **Index (staging area)** : zone intermédiaire où l'on prépare les modifications avant le commit 🔧
- **HEAD** : pointeur vers le dernier commit actif 📍

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Git (outil) vs GitHub / GitLab (plateformes)

Est-ce que Git et GitHub sont interchangeables ? ✋

❌ Pas du tout !

- **Git** est un outil local installé sur votre machine 💻
- **GitHub** ou **GitLab** sont des plateformes web permettant d'héberger vos dépôts Git à distance ☁️
- On peut utiliser Git **sans GitHub**, mais pas l'inverse

Local vs distant

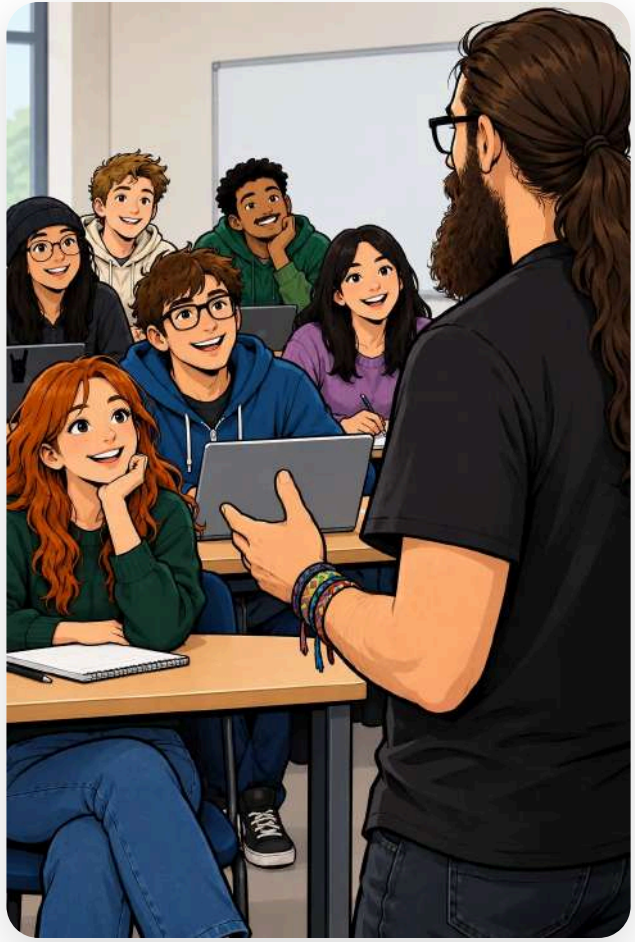
Quand travaille-t-on en local ? Et quand faut-il aller sur le distant ? ✋

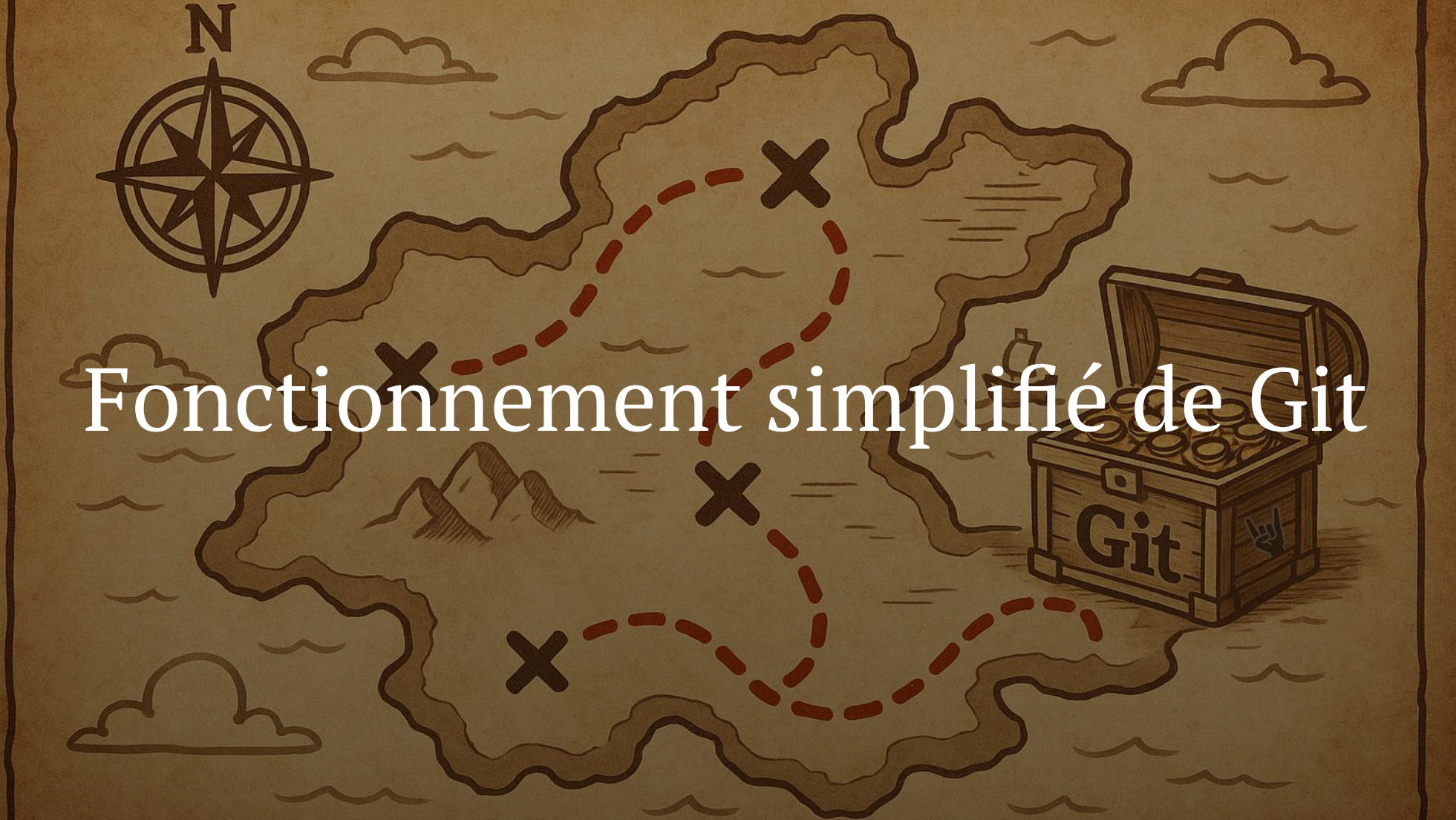
- **Local** : travail sur son propre ordinateur avec Git uniquement ⚙️
- **Distant** : interaction avec un serveur (comme GitHub) pour partager son travail avec d'autres 👥
- Git permet de gérer ces deux environnements et de synchroniser entre eux 🔄

Questions
Concepts fondamentaux

Vos questions ✋
Vos demandes d'ajout ✋

- ?





Fonctionnement simplifié de Git

Fonctionnement interne de Git

Git est-il un simple outil de comparaison de fichiers ? ✋

Non, Git ne se contente pas de stocker des différences :

- Il capture l'état complet des fichiers à chaque commit 📸
- Ces états sont appelés **snapshots**
- Cela garantit un historique fiable et rapide à manipuler ⚡

Git enregistre des snapshots

Quelle est la différence entre snapshot et différence (diff) ? ✋

- Les systèmes traditionnels enregistrent seulement les différences entre versions
- Git prend un **instantané complet** à chaque commit
- Si rien ne change, Git optimise en référençant le fichier précédent 🧠

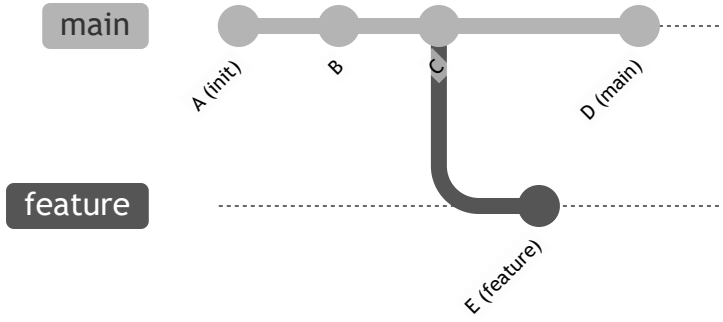
L’historique sous forme de graphe (DAG)

Comment Git organise-t-il les versions ? ✋

- Git structure l’historique comme un **graphe orienté acyclique** (DAG) 🧭
- Chaque commit pointe vers son ou ses parents
- Permet des fusions, branches, retours en arrière très flexibles

DAG dans Git

- Un **DAG (Directed Acyclic Graph)** est une structure qui représente l'historique Git :
  - **Graphe** : chaque commit est un **nœud** relié à ses **parents** 🔗
  - **Orienté** : chaque lien va d'un commit **vers ses prédécesseurs** 🔄
  - **Acyclique** : il **n'y a pas de boucle possible** ; l'historique ne revient jamais en arrière ⛔
- Exemple de graphe Git :



`C` a deux enfants : `D` (main) et `E` (feature)
Le graphe est orienté et ne boucle jamais ♉ ❌

Cette structure permet de gérer l'historique, les branches, les fusions avec efficacité et fiabilité.

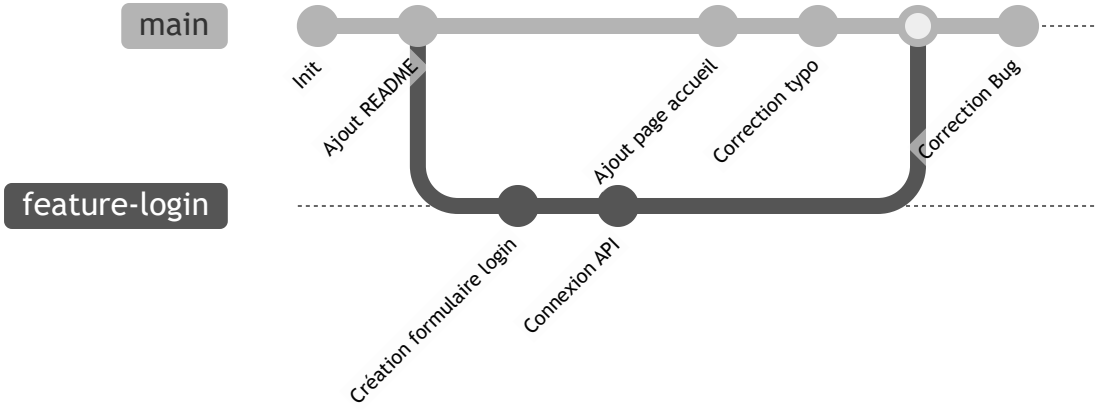
Visualisation simple : dossier, timeline, sauvegardes

Peut-on représenter Git de manière intuitive ? ✋

Imaginez :

- Un **dossier** contenant vos fichiers (votre projet)
- Une **timeline** avec des points (snapshots) correspondant aux commits 🕑
- Chaque commit contient un identifiant unique, un message, un auteur, une date et heure 📃

Exemple visuel : graphe Git simple



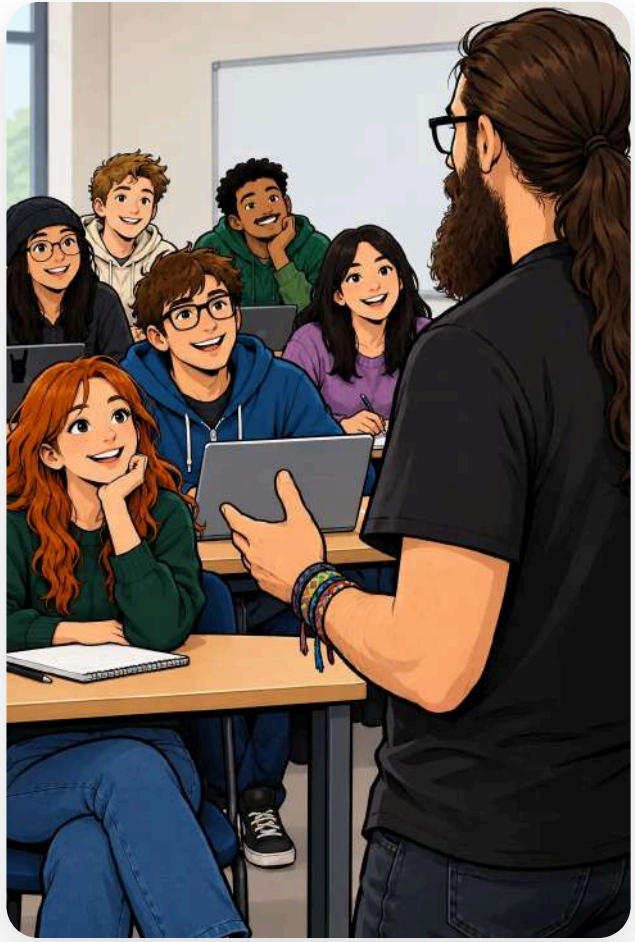
Questions

Fonctionnement simplifié de Git

Vos questions ✋

Vos demandes d'ajout ✋

- ?





Premiers pas avec Git

Bien démarrer avec Git

Quelles sont les premières étapes pour utiliser Git concrètement ? ✋

Avant de manipuler Git au quotidien, il faut :

- L'installer sur sa machine
- Le configurer avec ses informations personnelles
- Créer un dépôt local et y enregistrer ses premiers changements

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Installer Git (mention rapide)

Où trouver Git et comment l'installer ? ✋

- Git est téléchargeable depuis https://git-scm.com/
- Disponible sur Windows, macOS et Linux 🖥️
- La ligne de commande reste la base, c'est ce qu'on utilise dans ce cours 💡

Les interfaces graphiques les plus courantes :

Outil	Type	Lien
GitHub Desktop	App dédiée	desktop.github.com
GitKraken	App dédiée	gitkraken.com
IntelliJ / WebStorm	IDE	intégré nativement, onglet "Git"
VS Code	IDE	intégré nativement, onglet "Source Control"

> 🧔 J'utilise Git via mon IDE au quotidien. Mais les commandes restent indispensables : quand quelque chose se passe mal, c'est toujours en ligne de commande qu'on comprend vraiment ce qui se passe.

Configurer Git avec git config

Comment personnaliser son environnement Git ? ✋

- Configuration minimale à faire :

```
git config --global user.name "Votre Nom"
git config --global user.email "email@example.com"
```

- Cela permet d'identifier l'auteur de chaque commit 👄
- L'option `--global` applique ces réglages à tous les projets sur la machine

Créer un dépôt local

Comment initialise-t-on un nouveau projet Git ? ✋

- Se placer dans le dossier du projet avec le terminal

```
cd mon-projet
git init
```

- Cela crée un sous-dossier `.git` qui contient toute l'information de versionnage 📂
- Sur une plateforme comme GitHub ou GitLab, on peut aussi créer un dépôt depuis l'interface web et le récupérer ensuite.
- Sur certains OS, le dossier `.git` est caché par défaut, mais il est essentiel pour Git.

Les quatre zones de Git

Quand on tape `git add` *puis* `git commit`*, que se passe-t-il vraiment ?* ✋

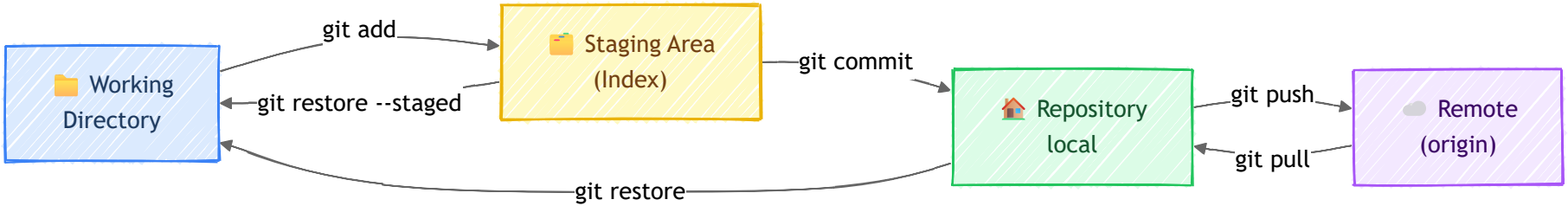
Git organise le travail en **quatre zones distinctes** :

- 📁 **Working Directory** : les fichiers tels que tu les vois sur ton disque
- 🗂️ **Staging Area (Index)** : la zone de préparation avant le commit
- 🏠 **Repository local** : l'historique Git stocké sur ta machine
- ☁️ **Remote** : le dépôt distant (GitHub, GitLab…)

> 🧔 Comprendre ces quatre zones, c'est comprendre 80% de Git. Tout le reste n'est que des commandes pour passer d'une zone à l'autre.

Le flux complet : de l'édition au partage

Comment un fichier modifié arrive-t-il sur GitHub ? ✋



Zoom : Working Directory

Qu'est-ce que le Working Directory concrètement ? ✋

- C'est simplement **le dossier de ton projet** sur ton ordinateur 💻
- Tu y crées, modifies, supprimes des fichiers normalement
- Git **observe** ces changements mais **ne les enregistre pas encore**
- Un fichier modifié ici est dit "**unstaged**" ou "**untracked**" s'il est nouveau

```
# Voir ce que Git observe dans ton Working Directory
git status
```

> 🧔 Modifier un fichier dans ton éditeur ne sauvegarde rien dans Git. Tu peux tout à fait travailler pendant une heure et n'avoir aucun commit. Ce n'est pas forcément un problème, mais il faut en être conscient.

Connaître l'état de ses fichiers avec git status

- `git status` affiche l'état de chaque fichier dans le Working Directory et la Staging Area 🔍
- Il indique clairement :
  - les fichiers **modifiés mais pas encore stagés** (unstaged)
  - les fichiers **stagés et prêts à être commités** (staged)
  - les fichiers **non suivis par Git** (untracked)

> 🤦 En pratique, je tape rarement `git status` car mon IDE me donne ces informations en temps réel, directement dans l'arborescence de fichiers. Mais en ligne de commande, c'est la première commande à utiliser par réflexe quand on est perdu.

```
git status
```

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   contact.txt       ← stagé ✅

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
        modified:   README.md         ← modifié, pas stagé ⚠️

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.txt                     ← inconnu de Git 👀
```

Zoom : Staging Area (Index)

À quoi ça sert vraiment de passer par le staging ? ✋

- La Staging Area est une **zone de préparation** : tu choisis **ce que tu veux inclure** dans ton prochain commit 🎯
- Elle permet de **sélectionner précisément** les changements à committer, même si tu as modifié 10 fichiers
- Un fichier ajouté au staging est dit "**staged**"

```
git add fichier.txt        # staging d'un fichier précis
git add .                  # staging de tous les fichiers modifiés
git status                 # voir ce qui est staged ou non
```

> 🧔 C'est l'outil qui te permet de faire des commits atomiques. Tu as modifié 3 fichiers pour deux raisons différentes ? Tu stages les uns, tu commites, puis tu stages les autres, tu commites. Propre et lisible.

Zoom : Repository local

Que contient exactement le repository local ? ✋

- C'est **l'historique complet du projet**, stocké dans le dossier `.git` sur ta machine 📦
- Chaque commit y est conservé avec : son identifiant, son auteur, sa date, son message, son snapshot
- Il est **entièrement autonome** : tu peux travailler sans connexion internet

```
git commit -m "feat: ajout du formulaire de contact"
git log          # consulter l'historique local
```

```
commit 3f6a2c1d... (HEAD -> main)
Author: Prénom Nom <email@example.com>
Date:   Mon Feb 24 10:32:00 2025

    feat: ajout du formulaire de contact

commit a1b2c3d4...
Author: Prénom Nom <email@example.com>
Date:   Sun Feb 23 18:14:00 2025

    Initial commit avec README
```

 C'est une des forces de Git face aux anciens systèmes centralisés : pas de réseau, pas de serveur, tout l'historique est là, sur ta machine.

Zoom : Remote (dépôt distant)

Quand et pourquoi envoyer son travail sur le remote ? ✋

- Le **remote** est une copie du dépôt hébergée sur un serveur (GitHub, GitLab, Bitbucket…) ☁️
- Il sert à :
  - **sauvegarder** son travail hors de sa machine 💾
  - **partager** avec d'autres collaborateurs 👥
  - servir de **référence commune** pour l'équipe

```
git push origin main    # envoyer ses commits locaux vers le remote
git pull origin main    # récupérer les commits du remote en local
```

> 🧑 Le remote ne se met jamais à jour tout seul. C'est toi qui décides quand tu partages ton travail. Ça veut dire qu'on peut très bien avoir 10 commits en local qui n'existent pas encore sur GitHub.

Récapitulatif : quelle commande pour quelle zone ?

Commande	De	Vers	Effet
`git status`	-	-	Indique l'état de chaque zone
`git add`	Working Directory	Staging Area	Prépare les fichiers pour le commit
`git commit`	Staging Area	Repository local	Enregistre un snapshot dans l'historique
`git push`	Repository local	Remote	Envoie les commits vers le serveur distant
`git pull`	Remote	Repository local + Working Directory	Récupère et intègre les nouveautés distantes
`git clone`	Remote	Repository local + Working Directory	Copie complète d'un dépôt distant en local
`git restore`	Staging Area	Working Directory	Annule les modifications non commitées

 Cette slide est idéale comme référence rapide. Tu peux l'imprimer ou la mettre en fond d'écran pour un accès rapide.

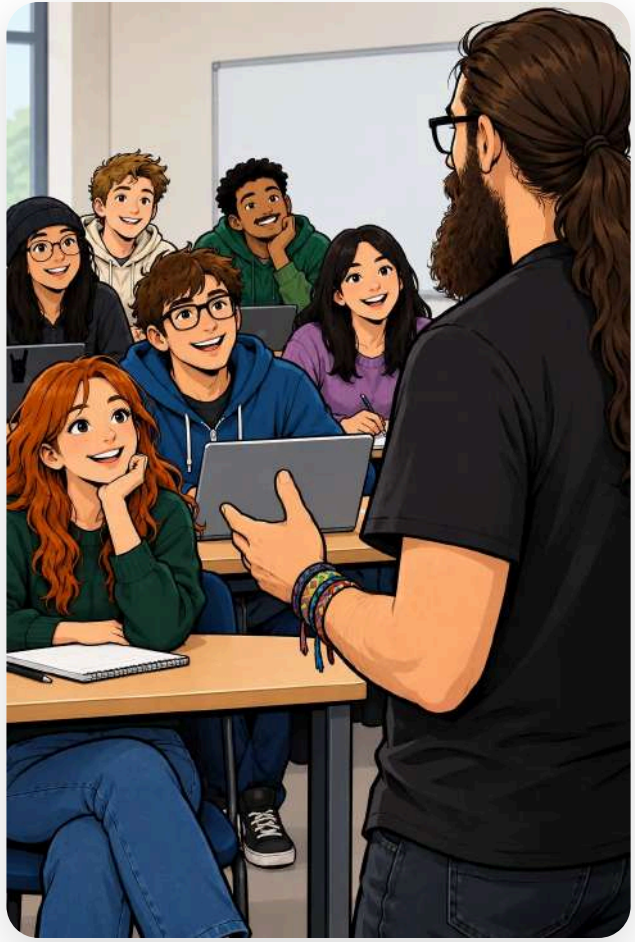
Questions

Premiers pas avec Git

Vos questions ✋

Vos demandes d'ajout ✋

- ?





Travailler avec un dépôt distant

Utiliser un dépôt distant

Git fonctionne-t-il uniquement en local ? ✋

- Un dépôt distant permet **sauvegarder son travail** sur un serveur externe 💾
- Il permet de **collaborer** avec d'autres personnes 👥
- Il agit comme un **point de convergence** des modifications
- On peut récupérer les changements des autres et envoyer les siens

Synchroniser avec un dépôt distant

Comment récupérer un projet existant depuis une plateforme comme GitHub ? ✋

- Commande :

```
git clone https://github.com/utilisateur/mon-projet.git
```

- Crée un dossier local avec tous les fichiers et l'historique complet 📁
- Le dépôt distant est automatiquement enregistré comme `origin`

Lier un dépôt distant à un projet local

Et si j'ai commencé localement, comment relier mon projet à GitHub ? ✋

- Commande :

```
git remote add origin https://github.com/utilisateur/mon-projet.git
```

- Cela ajoute un **lien** nommé `origin` pointant vers l'URL du dépôt distant
- Ce lien est nécessaire pour pouvoir `push` ou `pull`

L'origine d'`origine` ?

Pourquoi écrit-on `origin` dans `git push origin main` ? Qu'est-ce que c'est exactement ? ✋

- `origin` est un **alias** : un nom court qui pointe vers l'URL complète du dépôt distant 🚀
- Au lieu d'écrire :

```
git push https://github.com/utilisateur/mon-projet.git main
```

- On écrit simplement :

```
git push origin main
```

- `origin` n'est **pas un mot-clé Git** : c'est juste le nom donné par convention au premier remote ajouté

> 🐵 On pourrait l'appeler `github`, `serveur`, ou `bigbang`. Mais tout le monde utilise `origin` par convention, donc autant la respecter.

Création d'`origin`

Est-ce qu'on le crée manuellement, ou Git le définit tout seul ? ✋

Il est créé de **deux façons** selon comment on démarre le projet :

- **Via** `git clone` : Git crée `origin` automatiquement en pointant vers l'URL clonée

```
git clone https://github.com/utilisateur/mon-projet.git
# → origin est automatiquement défini
```

- **Via** `git remote add` : on le définit manuellement quand on part d'un projet local

```
git remote add origin https://github.com/utilisateur/mon-projet.git
# → on crée l'alias origin nous-mêmes
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Vérifier et gérer ses remotes

Comment savoir quels remotes sont configurés sur mon projet ? ✋

```
git remote -v
```

```
origin  https://github.com/utilisateur/mon-projet.git (fetch)
origin  https://github.com/utilisateur/mon-projet.git (push)
```

- La commande liste tous les remotes avec leur URL associée
- `(fetch)` : URL utilisée pour récupérer (`git pull`)
- `(push)` : URL utilisée pour envoyer (`git push`)
- Dans la plupart des cas, les deux sont identiques

> 🧑‍🚒 On peut tout à fait avoir plusieurs remotes sur un même projet. Par exemple `origin` pour son fork personnel et `upstream` pour le projet original open source. Mais pour débuter, un seul remote suffit largement.

Envoyer et récupérer les modifications

Une fois le lien établi, quelles commandes utiliser ? ✋

- **Envoyer** les commits locaux vers le dépôt distant :

```
git push origin main
```

- **Récupérer** les nouveautés depuis le dépôt distant :

```
git pull origin main
```

- Ces deux commandes assurent une bonne synchronisation 🔄

`git fetch` vs `git pull` : différence

`git pull` récupère les modifications distantes… mais que fait-il exactement ? ✋

- `git pull` est en réalité **deux commandes en une** : `git pull = git fetch + git merge`
- `git fetch` : télécharge les nouveautés du remote **sans toucher** à ton Working Directory
- `git merge` : intègre ces nouveautés dans ta branche locale

> `git pull` est pratique au quotidien, mais `git fetch` te donne le contrôle : tu récupères d'abord, tu regardes ce qui a changé, puis tu décides quand intégrer.

Quand utiliser `git fetch` plutôt que `git pull` ?

Est-ce qu'on a vraiment besoin de `git fetch` quand `git pull` fait tout ? ✋

- `git pull` : pour un usage simple et rapide, quand on est seul ou que l'équipe travaille sur des branches séparées
- `git fetch` : quand on veut **observer** les changements distants avant de les intégrer 🔍

```
git fetch origin          # récupère les nouveautés sans rien modifier
git log origin/main       # consulte l'historique distant
git merge origin/main     # intègre quand on est prêt
```

- Utile pour éviter des conflits surprises sur une branche partagée

> 🧔 Pour débuter, `git pull` suffit largement. Mais retiens que si un `git pull` te crée des conflits inattendus, `git fetch` + `git merge` te permettront de mieux comprendre ce qui se passe.

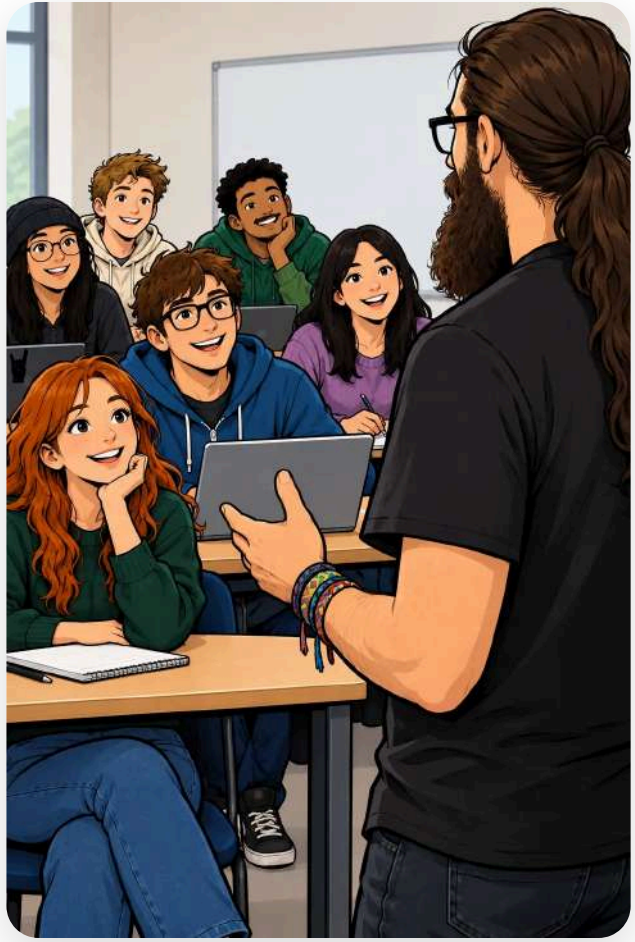
Questions

Travailler avec un dépôt distant

Vos questions ✋

Vos demandes d'ajout ✋

- ?





Branches

Utiliser des branches

Travailler à plusieurs ou tester une idée : comment ne pas tout casser ? ✋

- Les branches permettent de **travailler en parallèle** sans affecter la version principale 🌿
- On peut tester une fonctionnalité, corriger un bug, puis **fusionner** une fois validé ✅
- Pratique aussi pour organiser le développement entre membres d'une équipe 👥

Créer une branche

Comment démarrer un développement séparé ? ✋

- Commande :

```
git branch feature-x
```

- Cela crée une nouvelle branche appelée `feature-x` à partir de la branche actuelle
- Elle part du commit actuel mais ne l'affecte pas tant qu'on n'y travaille pas

Bascule entre les branches

Comment passer d'une branche à une autre ? ✋

- Ancienne syntaxe (toujours valide) :

```
git checkout feature-x
```

- Nouvelle syntaxe depuis Git 2.23 (2019), plus explicite :

```
git switch feature-x
```

- Pour créer une branche et basculer dessus en une seule commande :

```
git switch -c feature-x   # nouvelle syntaxe
git checkout -b feature-x  # ancienne syntaxe
```

> 🧔 `git checkout` fait trop de choses à la fois : changer de branche, restaurer des fichiers, détacher HEAD… `git switch` a été créé pour clarifier ça. Tu verras les deux en entreprise, il faut connaître les deux.

Fusionner une branche

Une fois satisfait, comment réintégrer les changements ? ✋

- Se placer sur la branche principale (`main`) puis :

```
git merge feature-x
```

- Cela ajoute les commits de `feature-x` dans `main`
- Si tout se passe bien, Git crée un commit de fusion automatique 🔗

Conflits simples : exemple visuel et explication

Que se passe-t-il si deux branches modifient la même ligne ? ✋

- Git signale un **conflit** qu'il ne peut résoudre automatiquement ❗
- Exemple typique : deux branches modifient le même paragraphe dans un fichier
- Git insère alors des marqueurs :

```
<<<<<<< HEAD
version principale
=======
version de feature-x
>>>>>>> feature-x
```

- Il faut éditer le fichier, choisir le bon contenu, puis :

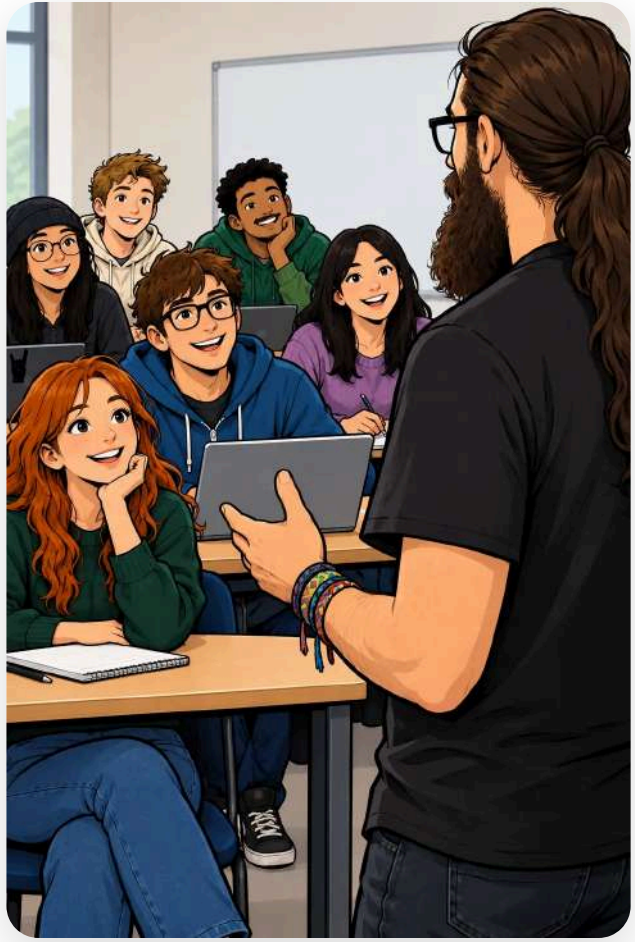
```
git add fichier.txt
git commit
```

> 🧔 Des outils graphiques permettent de faire ça facilement et rapidement.

Questions
Branches

Vos questions ✋
Vos demandes d'ajout ✋

- ?





Cas pratique guidé

Objectif de la mise en pratique

- Simuler un mini-projet à deux, en local puis avec GitLab ou GitHub 🌐
- Reproduire les étapes clés : init, add, commit, branch, merge, push 📋
- Comprendre les rôles respectifs du local et du distant

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Préparation : configuration initiale de Git

Quelles sont les configurations essentielles à faire avant de commencer ? ✋

- Configurer son identité pour que les commits soient bien identifiés :

```
git config --global user.name "Ton Nom"
git config --global user.email "ton.email@example.com"
```

- Définir `main` comme branche par défaut à la place de `master` :

```
git config --global init.defaultBranch main
```

- (Facultatif mais utile) Vérifier la configuration actuelle :

```
git config --list
```

- Ces réglages s'appliquent à tous vos futurs projets Git sur cette machine 🛠️

> 🧔 La nouvelle convention, depuis 2020, de nommage de la branche par défaut est `main` et non plus `master`.
> Je te mets la commande pour changer la branche par défaut au cas où.

Partie 1 : mise en place locale

Comment initier un projet Git dès le départ ? ✋

- Créer un nouveau dossier :

```
mkdir projet-demo
cd projet-demo
```

- Initialiser le dépôt Git :

```
git init
```

- Créer un fichier `README.md` :

```
echo "# Mon projet Git" > README.md
```

- Ajouter et committer :

```
git add README.md
git commit -m "Initial commit avec README"
```

Partie 2 : branche de fonctionnalité

Comment ajouter une fonctionnalité sans impacter le reste ? ✋

- Créer une branche :

```
git branch feature-contact
git checkout feature-contact
```

- Modifier un fichier ou en créer un nouveau :

```
echo "La page de contact !" > contact.txt
git add contact.txt
git commit -m "Ajout de la page de contact"
```

- Revenir sur `main` et fusionner :

```
git checkout main
git merge feature-contact
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Partie 3 : interaction avec GitLab ou GitHub

Comment publier son projet en ligne ? ✋

- Créer un **nouveau dépôt vide** sur GitLab ou GitHub depuis l'interface web
  - Fais attention de **ne pas cocher** l'option "Initialiser avec un README" car tu l'as déjà fait en local.
- Lier le dépôt distant :

```
git remote add origin https://github.com/mon-utilisateur/projet-demo.git
```

- Pousser les commits :

```
git push -u origin main
```

- Aller sur GitLab ou GitHub et constater les fichiers en ligne 🎉

> 🧔 J'ai laissé le résultat de ces commandes sur mon repositories **FramaGit**

> 🧔 Quand tu fais ton push, tu dois être authentifié avec ton compte GitLab ou GitHub.
> Tu auras certainement une étape de connexion à faire la première fois.

Partie 4 : en commençant par le dépôt distant

Comment démarrer un projet en ligne et le cloner ? ✋

- Créer un **nouveau dépôt vide** sur GitLab ou GitHub depuis l'interface web
  - Fais attention de **ne pas cocher** l'option "Initialiser avec un README" pour avoir les commande.
- Suis les instructions fournies pour cloner le dépôt.

> 🧔 J'ai créé un dépôt (quasi) vide pour que tu vois à quoi cela ressemble : **FramaGit**

Partie 5 : mise en situation collaborative

Comment travailler à deux sur le même projet sans se marcher dessus ? ✋

Dans cette partie, tu vas simuler une vraie collaboration en équipe :

- **Personne A** et **Personne B** ont tous les deux accès au même dépôt distant
- Chacun va cloner le dépôt, travailler sur **sa propre branche**, puis tout fusionner dans `main`
- C'est exactement comme ça que ça se passe en entreprise 🏢

> 🧔 Si tu travailles seul, fais les deux rôles à la suite sur la même machine dans deux dossiers séparés.

Partie 6 : Les deux : cloner le dépôt

Comment récupérer le projet en local pour commencer à travailler ? ✋

Les deux collaborateurs clonent le même dépôt distant sur leur machine :

```
# Personne A
git clone https://github.com/utilisateur/initiation-git.git dossier-personne-a
cd dossier-personne-a
```

```
# Personne B (dans un dossier séparé)
git clone https://github.com/utilisateur/initiation-git.git dossier-personne-b
cd dossier-personne-b
```

- Le nom du dossier à la fin de `git clone` est optionnel, il permet ici de différencier les deux clones sur une même machine
- Les deux partent du même état : même historique, même contenu 📁

Partie 7 : chacun travaille sur sa branche

Comment travailler sans impacter le travail de l'autre ? ✋

```
# La personne A crée sa branche et y ajoute sa contribution :
git branch feature-accueil
git checkout feature-accueil
echo "# Page d'accueil" > accueil.txt
echo "Bienvenue sur le projet !" >> accueil.txt
git add accueil.txt
git commit -m "feat: ajout de la page d'accueil"
git push origin feature-accueil
```

```
# En parallèle, la personne B fait de même sur sa branche :
git branch feature-contact
git checkout feature-contact
echo "# Page de contact" > contact-v2.txt
echo "Email : contact@exemple.com" >> contact-v2.txt
git add contact-v2.txt
git commit -m "feat: ajout de la page de contact"
git push origin feature-contact
```

> 🧔 Les deux travaillent en même temps, chacun sur sa branche, sans se gêner 👐

Partie 8 : fusionner les deux branches dans `main`

Comment rassembler le travail des deux collaborateurs ? ✋

L'un des deux (ici A) se charge de fusionner les deux branches dans `main` :

```
# Se placer sur main et récupérer les dernières nouveautés
git checkout main
git pull origin main
```

```
# Fusionner la branche de A puis celle de B
git merge feature-accueil
git merge feature-contact
```

```
# Pousser le résultat sur le dépôt distant
git push origin main
```

> 🧔 En entreprise, cette étape de fusion passe souvent par une Pull Request (GitHub) ou Merge Request (GitLab) : ça permet à un pair de relire avant de fusionner. C'est la bonne pratique à retenir pour la suite.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Partie 9 : vérification finale

Comment s'assurer que tout le travail des deux est bien intégré ? ✋

```
git log --oneline
```

```
f7a3c2e (HEAD -> main, origin/main) Merge branch 'feature-contact'
b4d1e9f Merge branch 'feature-accueil'
c2a8b3d feat: ajout de la page de contact
e1f4d2a feat: ajout de la page d'accueil
a1b2c3d Initial commit avec README
```

```
ls
```

```
README.md   accueil.txt   contact-v2.txt
```

- L'historique montre les commits des deux contributeurs ✅
- Les fichiers des deux sont présents dans `main` ✅
- Le dépôt distant est à jour pour tout le monde ✅

Conclusion

- Félicitations, tu as réussi à mettre en place un projet Git complet et collaboratif ! 🎉
- Tu as appris à initier un dépôt, créer des branches, fusionner des modifications et interagir avec GitLab ou GitHub
- Tu sais maintenant travailler en parallèle sur des branches séparées et intégrer les contributions de chacun 🤝
- N'hésite pas à pratiquer régulièrement : Git devient naturel avec l'habitude 💪

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

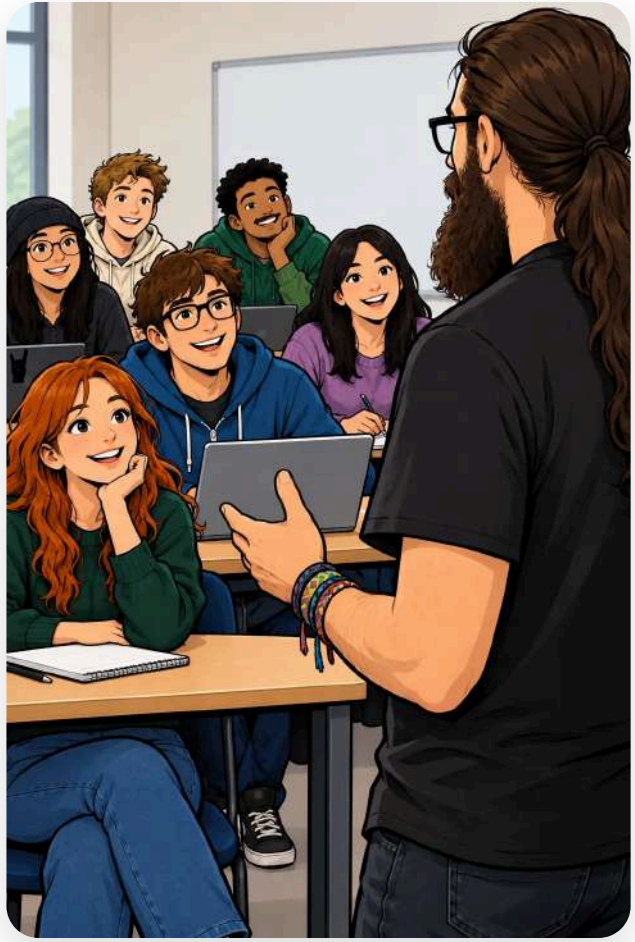
Questions

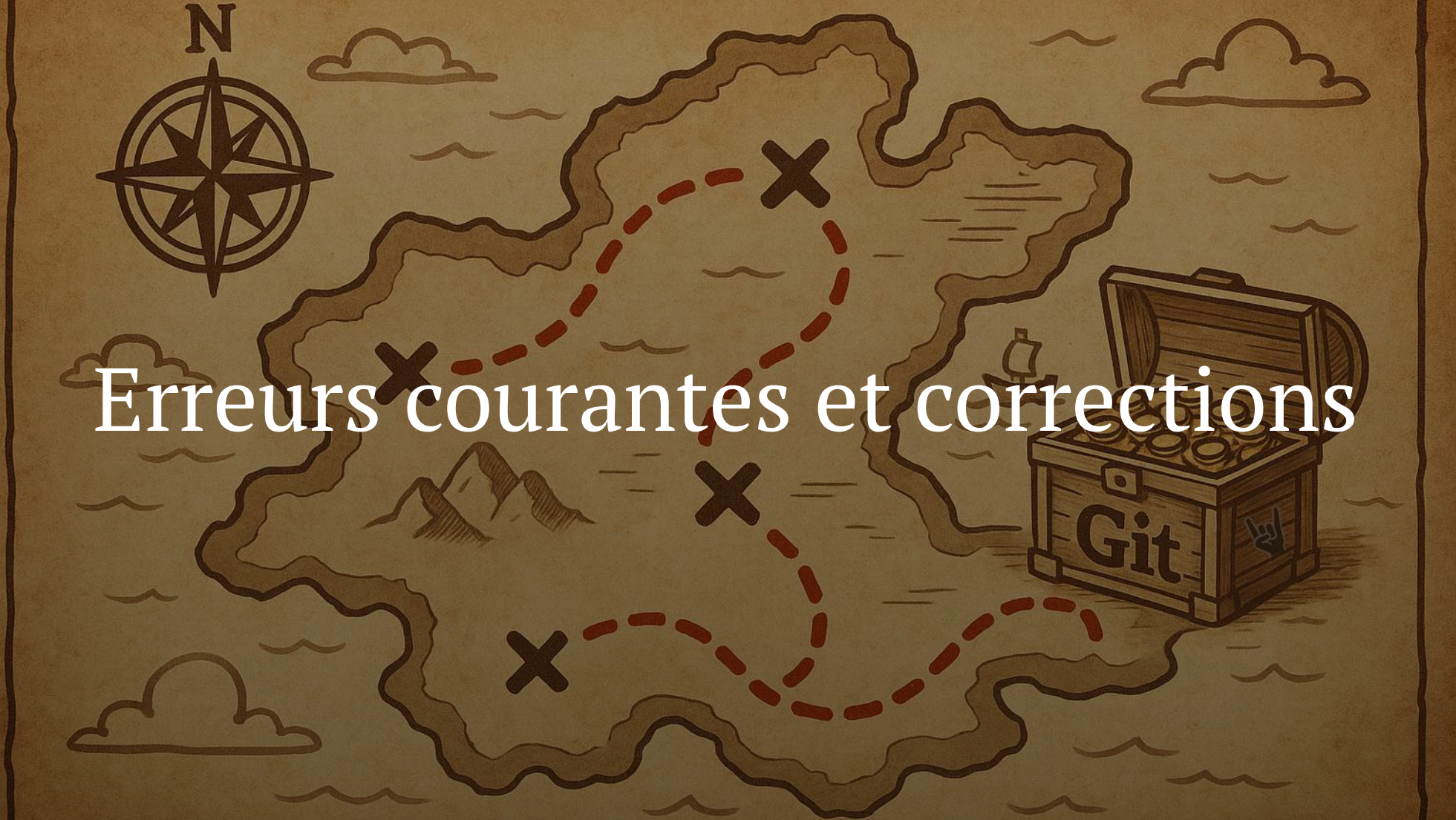
Cas pratique guidé

Vos questions ✋

Vos demandes d'ajout ✋

- ?





Erreurs courantes et corrections

Quand Git résiste… c’est souvent pour une bonne raison

- Git est strict, et c’est une qualité : il ne laisse pas passer les erreurs silencieusement 🛡️
- La plupart des messages d’erreur font peur au premier abord, mais ils sont souvent très explicites
- Ce chapitre couvre les situations les plus fréquentes que tu vas rencontrer en pratique

> 🧔 J’ai fait toutes ces erreurs. Plusieurs fois. Et je les refais encore de temps en temps. Ce qui change avec l’expérience, c’est qu’on les reconnaît plus vite.

Erreur 1 : j'ai oublié un fichier dans mon dernier commit

Tu viens de committer, et tu réalises qu'il manque un fichier. Que faire ? ✋

- Ajouter le fichier oublié au staging :

```
git add fichier-oublie.txt
```

- Amender le dernier commit sans changer son message :

```
git commit --amend --no-edit
```

- Ou amender en modifiant aussi le message :

```
git commit --amend -m "Nouveau message corrigé"
```

> 🧔 `--amend` réécrit le dernier commit. À n'utiliser que si tu n'as pas encore pushé, sinon tu vas créer des incohérences avec le dépôt distant.

Erreur 2 : j'ai commité sur la mauvaise branche

Tu réalises que ton commit est sur `main` alors qu'il aurait dû être sur une branche feature. ✋

- Créer la branche qui aurait dû accueillir le commit :

```
git branch feature-x
```

- Revenir en arrière sur `main` sans perdre le commit :

```
git reset HEAD~1
```

- Basculer sur la bonne branche et committer :

```
git checkout feature-x
git add .
git commit -m "feat: le bon commit sur la bonne branche"
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Erreur 3 : je veux annuler un commit déjà pushé

Tu as pushé un commit qui ne devrait pas être là. Comment l'annuler proprement ? ✋

- Ne **jamais** utiliser `git reset` sur un commit déjà pushé sur une branche partagée ❌
- Utiliser `git revert` à la place : il crée un nouveau commit qui annule le précédent, sans réécrire l'historique ✅

```
git revert HEAD
```

- Puis pousser le commit d'annulation :

```
git push origin main
```

```
Avant : A → B → C (commit à annuler)
Après : A → B → C → D (D annule C)
```

Erreur 4 : j'ai des modifications que je ne veux pas committer maintenant

Tu dois changer de branche en urgence, mais tu as des modifications en cours non commitées. ✋

- Mettre ses modifications de côté temporairement avec `git stash` :

```
git stash
```

- Changer de branche, faire ce qu'il y a à faire, puis revenir :

```
git checkout autre-branche
# ... travail urgent ...
git checkout ma-branche
```

- Récupérer ses modifications mises de côté :

```
git stash pop
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Erreur 5 : git push est refusé

Tu fais un `git push` et Git te répond `rejected`. Que se passe-t-il ? ✋

```
 ! [rejected] main -> main (fetch first)
error: failed to push some refs to '...'
hint: Updates were rejected because the remote contains work that you do not have locally.
```

- Quelqu'un d'autre a pushé sur la même branche depuis ton dernier `pull` 🚦
- Git refuse pour ne pas écraser son travail
- La solution : récupérer d'abord, puis repusher :

```
git pull origin main
# résoudre les éventuels conflits
git push origin main
```

> 🧔 C'est le message d'erreur numéro 1 quand on travaille en équipe. Il ne faut surtout pas forcer avec `git push --force` sur une branche partagée, tu écrases le travail des autres.

Erreur 6 : je suis perdu, je ne sais plus où j'en suis

Trop de commandes, trop de branches, tu ne sais plus dans quel état est ton dépôt. ✋

Trois commandes pour reprendre le contrôle :

- Voir l'état des fichiers :

```
git status
```

- Voir l'historique des commits :

```
git log --oneline
```

- Voir toutes les branches et où on se trouve :

```
git branch -a
```

> 🧔 Quand je suis perdu, je commence toujours par `git status`. Dans 90% des cas, Git me dit exactement ce que je dois faire dans les lignes `hint:` qui suivent le message.

Récapitulatif : quelle commande pour quelle erreur ?

Situation	Commande
Oublié un fichier dans le dernier commit (pas encore pushé)	`git commit --amend`
Commité sur la mauvaise branche (pas encore pushé)	`git reset HEAD~1` puis changer de branche
Annuler un commit déjà pushé	`git revert HEAD`
Modifications en cours à mettre de côté	`git stash` / `git stash pop`
Push refusé car le distant a avancé	`git pull` puis `git push`
Perdu, ne sait plus où on en est	`git status`, `git log --oneline`, `git branch -a`



Comme la slide des commandes, cette slide est idéale comme référence rapide.

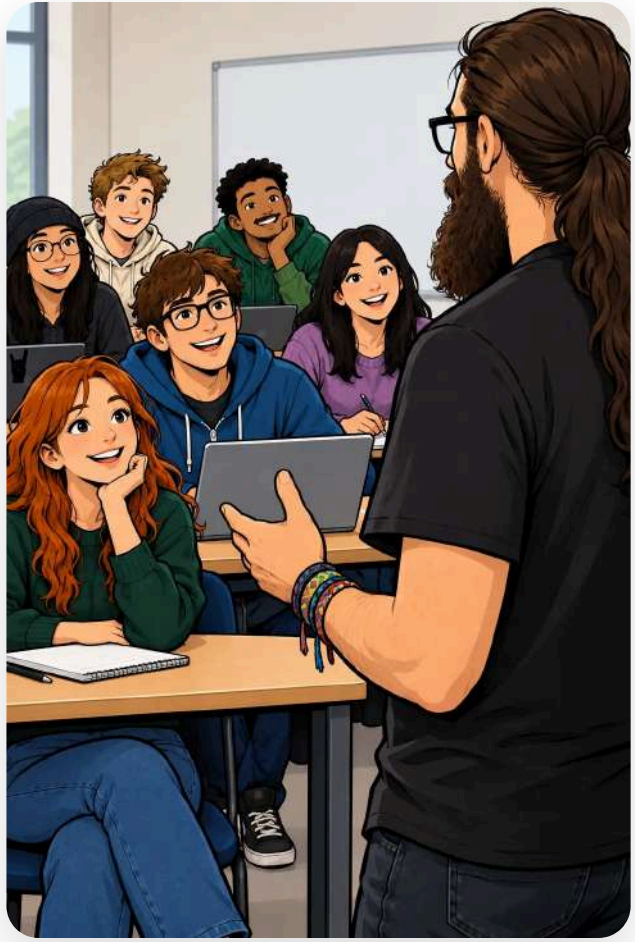
Questions

Erreurs courantes et corrections

Vos questions ✋

Vos demandes d'ajout ✋

- ?





Bonnes pratiques

Pourquoi adopter des bonnes pratiques avec Git ?

- Git est puissant, mais sans discipline, l'historique devient illisible et difficile à maintenir 📉
- Des conventions simples rendent la collaboration plus fluide, surtout en équipe 👥
- Suivre de bonnes pratiques permet de faciliter :
  - les revues de code 🧐
  - les retours en arrière ⏪
  - la compréhension du projet par de nouveaux arrivants 🚀

Faire des commits clairs, fréquents et atomiques

- **Commits clairs** : un message doit expliquer ce qui a changé et pourquoi 💬
  - Exemple : `Fix: corrige le calcul du total dans panier.js`
- **Commits fréquents** : éviter d'attendre la fin du projet pour tout committer 🔁
  - Cela permet d'identifier rapidement les erreurs
- **Commits atomiques** : un commit = un changement logique
  - Mauvais exemple : `Ajout de login + correction bug affichage + mise à jour README`
  - Bon exemple : trois commits séparés, chacun pour un objectif précis

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Conventional Commits : une convention pour aller plus loin

Comment aller plus loin qu'un simple message de commit clair ? ✋

Conventional Commits est une convention de nommage des messages de commit : `<type>(<portée optionnelle>): <description>`

```
feat(auth): ajout de la connexion par email
fix(panier): corrige le calcul du total
docs: mise à jour du README
```

Type	Usage
`feat`	Nouvelle fonctionnalité
`fix`	Correction de bug
`docs`	Documentation uniquement
`chore`	Tâche technique sans impact fonctionnel
`refactor`	Réécriture sans changement de comportement

> 🥷 En entreprise, cette convention est souvent imposée. Elle permet d'automatiser la génération de changelogs et de versionner proprement. Même si tu ne l'appliques pas partout, la connaître te rend crédible en équipe.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Utiliser .gitignore

Pourquoi certains fichiers ne doivent-ils pas être suivis ? ✋

- Certains fichiers ne sont pas utiles au projet partagé :
  - fichiers temporaires de cache ou de logs,
  - fichiers de config locale,
  - fichiers contenant des secrets (comme `.env`),
  - etc.
- `.gitignore` permet d'éviter de committer accidentellement ces fichiers 🚫
- Exemple :

```
echo ".env" >> .gitignore
```

- Cela allège le dépôt et sécurise le code 💡

> 🧔 Quand tu crées un projet avec un IDE ou un framework, il génère souvent un `.gitignore` adapté. Vérifie-le et ajuste-le si nécessaire.

Règle du fichier .gitignore

Symbole ou motif	Exemple	Explication
`/` (début de ligne)	`/config/`	Spécifie un chemin depuis la racine du dépôt
`**`	`**/cache/`	Correspond à n'importe quel niveau de sous-répertoire
`!` (début de ligne)	`!important.txt`	Annule une règle précédente pour **inclure** un fichier/dossier spécifique
`#` (début de ligne)	`# Explication`	Sert à commenter une ligne et documenter les règles dans le `.gitignore`
`/` (fin de ligne)	`temp/` ou `/config/`	Indique que l'on cible un **dossier**

 Il est possible de cumuler les règles pour affiner le comportement, mais fais attention, ça devient vite compliqué.

Utiliser les branches pour isoler les changements

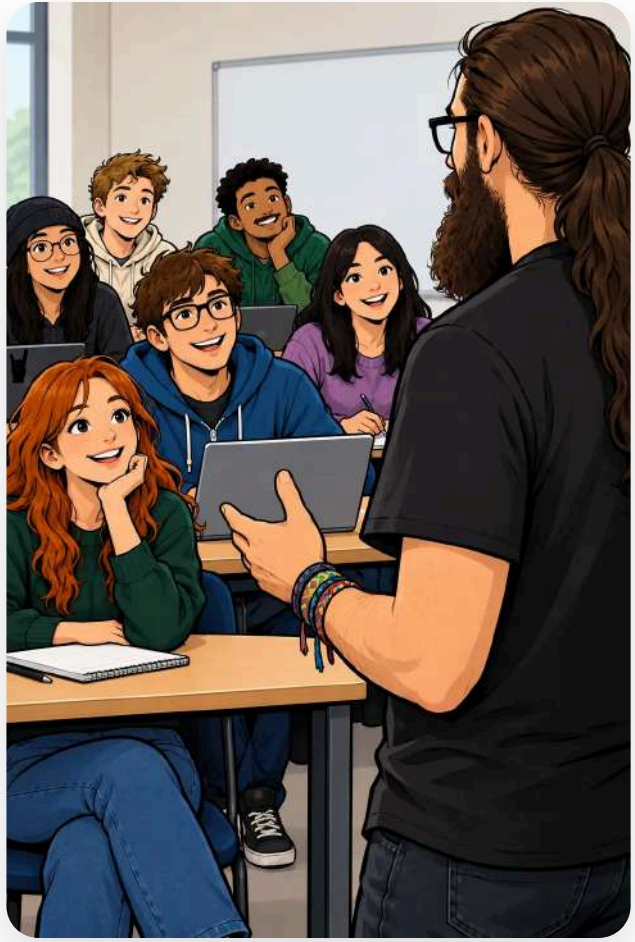
Quand créer une branche, et pourquoi ? ✋

- Chaque branche peut représenter :
  - Une nouvelle **fonctionnalité** (ex: `feature/search-bar`)
  - Une **correction de bug** (ex: `fix/button-alignment`)
  - Une **expérimentation temporaire**
- Travailler en branche permet de :
  - éviter de casser `main`
  - tester librement, sans risque pour les autres développeurs
  - faciliter les revues de code avant fusion 🔍

Questions
Bonnes pratiques

Vos questions ✋
Vos demandes d'ajout ✋

- ?





Glossaire

Vocabulaire essentiel de Git

- **Git** : système de gestion de versions distribué permettant de suivre les modifications et de collaborer efficacement.
- **Dépôt (repository)** : dossier contenant les fichiers versionnés et les données de suivi gérées par Git.
- **GitHub / GitLab** : plateformes en ligne permettant d'héberger des dépôts Git et de collaborer à distance.
- **Commit** : enregistrement d'un ensemble de modifications, avec un message descriptif.
- **Commit clair / atomique** : message lisible sur un changement précis et isolé.
- **Conventional Commits** : convention de nommage des commits.
- **Branche** : piste de développement indépendante permettant de développer sans perturber la branche principale.
- **Fusion (merge)** : réunir deux branches en une seule, en combinant leurs modifications.
- **Conflit** : désaccord entre des modifications concurrentes entre branches, nécessitant une résolution manuelle.
- **HEAD** : pointeur vers le dernier commit actif dans la branche courante.
- **Working Directory** : dossier de travail contenant les fichiers tels qu'ils sont sur le disque.
- **Index / Staging area** : zone intermédiaire où l'on place les fichiers à inclure dans le prochain commit.
- **Snapshot** : état complet du projet à un instant donné, capturé par un commit.
- **Différence (diff)** : variation entre deux versions d'un fichier.
- **origin** : alias par défaut pointant vers l'URL du dépôt distant.
- **remote** : dépôt distant hébergé sur un serveur (GitHub, GitLab…).
- **.gitignore** : fichier listant les fichiers ou dossiers à exclure du suivi Git.
- **main / master** : nom de la branche principale. `main` est recommandé depuis 2020.
- **DAG (Directed Acyclic Graph)** : structure de graphe orienté acyclique utilisée par Git pour représenter l'historique des commits.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Commandes Git courantes (1/2)

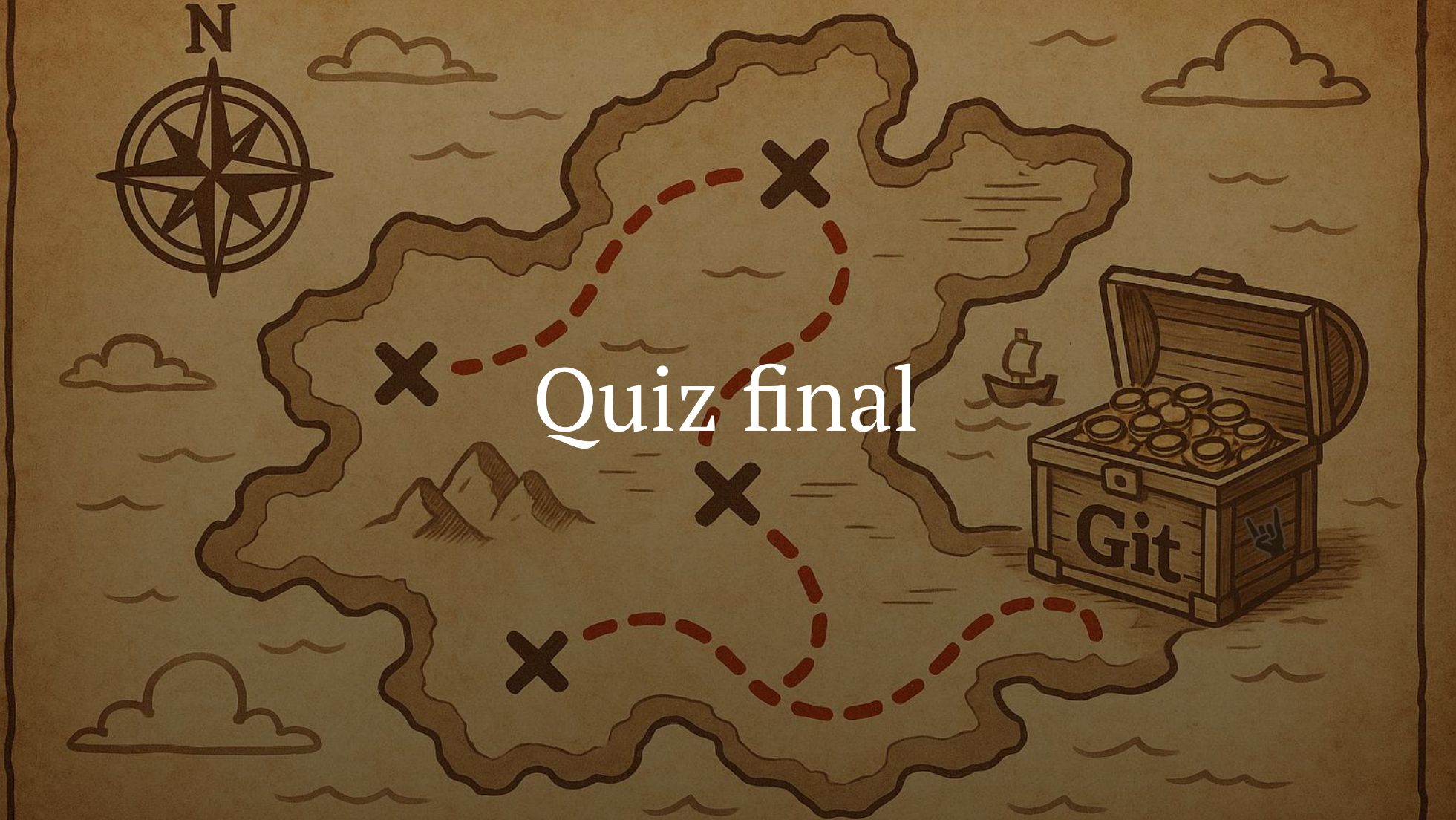
- **Mise en place**
  - **git init** : initialise un nouveau dépôt Git local.
  - **git config** : configure l'identité de l'utilisateur et d'autres options Git.
  - **git clone** : copie un dépôt distant en local.
- **Suivi des fichiers**
  - **git status** : affiche l'état des fichiers dans le Working Directory et la Staging Area.
  - **git add** : ajoute un fichier à l'index (staging area).
  - **git commit** : enregistre un snapshot des fichiers stagés dans l'historique local.
  - **git log** : affiche l'historique des commits.
- **Branches**
  - **git branch** : crée ou liste des branches.
  - **git switch** : change de branche (syntaxe moderne, depuis Git 2.23).
  - **git checkout** : change de branche ou restaure des fichiers (ancienne syntaxe, toujours valide).
  - **git merge** : fusionne une branche dans la branche active.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Commandes Git courantes (2/2)

- **Dépôt distant**
  - **git remote add** : ajoute un lien nommé vers un dépôt distant.
  - **git remote -v** : liste les remotes configurés et leurs URLs.
  - **git push** : envoie les commits locaux vers le dépôt distant.
  - **git fetch** : récupère les nouveautés du distant sans modifier le Working Directory.
  - **git pull** : récupère et intègre les nouveautés du distant (`fetch` + `merge`).
- **Corrections**
  - **git commit --amend** : modifie le dernier commit (avant push uniquement).
  - **git reset HEAD~1** : annule le dernier commit en conservant les modifications.
  - **git revert** : crée un commit qui annule un commit existant, sans réécrire l'historique.
  - **git restore** : annule des modifications dans le Working Directory ou la Staging Area.
  - **git stash** : met de côté des modifications non commitées temporairement.
  - **git stash pop** : restaure les modifications mises de côté avec `git stash` .

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**



Quiz final

Objectif du quiz

- Tester vos connaissances sur les concepts de base de Git et les bonnes pratiques
- Réviser les commandes essentielles et leur utilisation
- Comprendre les principes de gestion de versions et de collaboration
- Identifier les erreurs courantes et les éviter

> 🧔 Teste tes connaissances fraichement acquises ! Ce quiz est là pour t'aider à consolider tes bases en Git. Réponds aux questions et vérifie tes réponses à la fin.

Questions : Générales

1. Git est principalement :
   - ☐ Un hébergeur de fichiers
   - ☐ Un langage de script
   - ☐ Un gestionnaire de versions distribué
   - ☐ Une base de données
2. GitHub est :
   - ☐ Un logiciel à installer
   - ☐ Une interface graphique pour Git
   - ☐ Une plateforme web pour héberger du code Git
   - ☐ Une commande Git
3. Qui a créé Git ?
   - ☐ Bill Gates
   - ☐ Linus Torvalds
   - ☐ Dennis Ritchie
   - ☐ Steve Jobs
4. Un dépôt Git contient :
   - ☐ Un seul fichier
   - ☐ L'historique des modifications
   - ☐ Des images uniquement
   - ☐ Un système de messagerie
5. Git permet de :
   - ☐ Modifier des fichiers sans sauvegarde
   - ☐ Travailler à plusieurs en toute sécurité.
   - ☐ Supprimer des logiciels
   - ☐ Gérer des bases de données
6. L'historique Git est :
   - ☐ Linéaire
   - ☐ Circulaire
   - ☐ Un graphe orienté acyclique (DAG)
   - ☐ Aléatoire

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Réponses : Générales

1. Git est principalement :
   - ❌ ~~Un hébergeur de fichiers~~
   - ❌ ~~Un langage de script~~
   - ✅ Un gestionnaire de versions distribué
   - ❌ ~~Une base de données~~
2. GitHub est :
   - ❌ ~~Un logiciel à installer~~
   - ❌ ~~Une interface graphique pour Git~~
   - ✅ Une plateforme web pour héberger du code Git
   - ❌ ~~Une commande Git~~
3. Qui a créé Git ?
   - ❌ ~~Bill Gates~~
   - ✅ Linus Torvalds
   - ❌ ~~Dennis Ritchie~~
   - ❌ ~~Steve Jobs~~
4. Un dépôt Git contient :
   - ❌ ~~Un seul fichier~~
   - ✅ L'historique des modifications
   - ❌ ~~Des images uniquement~~
   - ❌ ~~Un système de messagerie~~
5. Git permet de :
   - ❌ ~~Modifier des fichiers sans sauvegarde~~
   - ✅ Travailler à plusieurs en toute sécurité.
   - ❌ ~~Supprimer des logiciels~~
   - ❌ ~~Gérer des bases de données~~
6. L'historique Git est :
   - ❌ ~~Linéaire~~
   - ❌ ~~Circulaire~~
   - ✅ Un graphe orienté acyclique (DAG)
   - ❌ ~~Aléatoire~~

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Questions : Concepts de base

1. Que signifie "committer" ?
   - ☐ Créer une branche
   - ☐ Fusionner un fichier
   - ☐ Enregistrer un ensemble de modifications
   - ☐ Supprimer un fichier
2. Une branche est :
   - ☐ Une sauvegarde
   - ☐ Une copie de l'historique principal pour travail
   - ☐ Une image du disque
   - ☐ Une option de sécurité
3. L'index (staging area) est :
   - ☐ L'historique des commits
   - ☐ La liste des utilisateurs
   - ☐ Une zone tampon avant commit
   - ☐ Un outil d'analyse
4. HEAD désigne :
   - ☐ Le dernier utilisateur
   - ☐ Le dernier fichier modifié
   - ☐ Le dernier commit pointé
   - ☐ Le nom du projet
5. Un commit contient :
   - ☐ Un identifiant unique
   - ☐ Une image
   - ☐ Un plugin
   - ☐ Un mot de passe
6. Une "timeline" Git est :
   - ☐ Un rapport imprimé
   - ☐ Une chronologie des commits
   - ☐ Une feuille de style
   - ☐ Une sauvegarde manuelle

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Réponses : Concepts de base

1. Que signifie "committer" ?
   - ❌ ~~Créer une branche~~
   - ❌ ~~Fusionner un fichier~~
   - ✅ Enregistrer un ensemble de modifications
   - ❌ ~~Supprimer un fichier~~
2. Une branche est :
   - ❌ ~~Une sauvegarde~~
   - ✅ Une copie de l'historique principal pour travail
   - ❌ ~~Une image du disque~~
   - ❌ ~~Une option de sécurité~~
3. L'index (staging area) est :
   - ❌ ~~L'historique des commits~~
   - ❌ ~~La liste des utilisateurs~~
   - ✅ Une zone tampon avant commit
   - ❌ ~~Un outil d'analyse~~
4. HEAD désigne :
   - ❌ ~~Le dernier utilisateur~~
   - ❌ ~~Le dernier fichier modifié~~
   - ✅ Le dernier commit pointé
   - ❌ ~~Le nom du projet~~
5. Un commit contient :
   - ✅ Un identifiant unique
   - ❌ ~~Une image~~
   - ❌ ~~Un plugin~~
   - ❌ ~~Un mot de passe~~
6. Une "timeline" Git est :
   - ❌ ~~Un rapport imprimé~~
   - ✅ Une chronologie des commits
   - ❌ ~~Une feuille de style~~
   - ❌ ~~Une sauvegarde manuelle~~

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Questions : Commandes Git

1. `git init` :
   - ☐ Crée un fichier
   - ☐ Crée un dépôt local
   - ☐ Efface tout
   - ☐ Met à jour un fichier
2. `git add` :
   - ☐ Ajoute des utilisateurs
   - ☐ Ajoute des fichiers au staging
   - ☐ Installe Git
   - ☐ Renomme un fichier
3. `git commit -m` :
   - ☐ Supprime un commit
   - ☐ Sauvegarde les modifications ajoutées
   - ☐ Crée une branche
   - ☐ Pousse vers GitHub
4. `git clone` :
   - ☐ Copie un dépôt distant localement
   - ☐ Renomme un fichier
   - ☐ Efface les commits
   - ☐ Crée une branche
5. `git push origin main` :
   - ☐ Envoie les commits locaux sur le dépôt distant
   - ☐ Télécharge les commits distants
   - ☐ Crée un nouveau dépôt GitHub
   - ☐ Initialise un fichier README
6. `git merge` :
   - ☐ Supprime une branche
   - ☐ Crée une nouvelle branche
   - ☐ Fusionne deux branches
   - ☐ Réinitialise le dépôt

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Réponses : Commandes Git

1. `git init` :
   - ❌ ~~Crée un fichier~~
   - ✅ Crée un dépôt local
   - ❌ ~~Efface tout~~
   - ❌ ~~Met à jour un fichier~~
2. `git add` :
   - ❌ ~~Ajoute des utilisateurs~~
   - ✅ Ajoute des fichiers au staging
   - ❌ ~~Installe Git~~
   - ❌ ~~Renomme un fichier~~
3. `git commit -m` :
   - ❌ ~~Supprime un commit~~
   - ✅ Sauvegarde les modifications ajoutées
   - ❌ ~~Crée une branche~~
   - ❌ ~~Pousse vers GitHub~~
4. `git clone` :
   - ✅ Copie un dépôt distant localement
   - ❌ ~~Renomme un fichier~~
   - ❌ ~~Efface les commits~~
   - ❌ ~~Crée une branche~~
5. `git push origin main` :
   - ✅ Envoie les commits locaux sur le dépôt distant
   - ❌ ~~Télécharge les commits distants~~
   - ❌ ~~Crée un nouveau dépôt GitHub~~
   - ❌ ~~Initialise un fichier README~~
6. `git merge` :
   - ❌ ~~Supprime une branche~~
   - ❌ ~~Crée une nouvelle branche~~
   - ✅ Fusionne deux branches
   - ❌ ~~Réinitialise le dépôt~~

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Questions : Bonnes pratiques

1. Un bon message de commit doit :
   - ☐ Être court et clair
   - ☐ Contenir des emojis seulement
   - ☐ Être identique à chaque fois
   - ☐ Être optionnel
2. Un commit atomique signifie :
   - ☐ Qu'il contient tous les changements d'un projet
   - ☐ Qu'il traite une seule chose à la fois
   - ☐ Qu'il est automatique
   - ☐ Qu'il ne peut pas être modifié
3. `.gitignore` permet :
   - ☐ D'ignorer les erreurs
   - ☐ D'exclure certains fichiers du suivi
   - ☐ D'annuler un commit
   - ☐ De renommer un dossier
4. Il est recommandé de committer :
   - ☐ Une fois par mois
   - ☐ Après chaque modification logique
   - ☐ Quand on s'en souvient
   - ☐ À la fin du projet uniquement
5. Nommer une branche `feature/` indique :
   - ☐ Une branche pour tester une erreur
   - ☐ Une branche dédiée à une nouvelle fonctionnalité
   - ☐ Une branche pour supprimer des fichiers
   - ☐ Une branche système
6. Un commit fourre-tout :
   - ☐ Est plus facile à lire
   - ☐ Peut rendre l'historique confus
   - ☐ Est toujours préféré
   - ☐ Est obligatoire

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Réponses : Bonnes pratiques

1. Un bon message de commit doit :
   - ✅ Être court et clair
   - ❌ ~~Contenir des emojis seulement~~
   - ❌ ~~Être identique à chaque fois~~
   - ❌ ~~Être optionnel~~
2. Un commit atomique signifie :
   - ❌ ~~Qu'il contient tous les changements d'un projet~~
   - ✅ Qu'il traite une seule chose à la fois
   - ❌ ~~Qu'il est automatique~~
   - ❌ ~~Qu'il ne peut pas être modifié~~
3. `.gitignore` permet :
   - ❌ ~~D'ignorer les erreurs~~
   - ✅ D'exclure certains fichiers du suivi
   - ❌ ~~D'annuler un commit~~
   - ❌ ~~De renommer un dossier~~
4. Il est recommandé de committer :
   - ❌ ~~Une fois par mois~~
   - ✅ Après chaque modification logique
   - ❌ ~~Quand on s'en souvient~~
   - ❌ ~~À la fin du projet uniquement~~
5. Nommer une branche `feature/` indique :
   - ❌ ~~Une branche pour tester une erreur~~
   - ✅ Une branche dédiée à une nouvelle fonctionnalité
   - ❌ ~~Une branche pour supprimer des fichiers~~
   - ❌ ~~Une branche système~~
6. Un commit fourre-tout :
   - ❌ ~~Est plus facile à lire~~
   - ✅ Peut rendre l'historique confus
   - ❌ ~~Est toujours préféré~~
   - ❌ ~~Est obligatoire~~

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Questions : Erreurs courantes

1. `git commit --amend` s'utilise :
   - ☐ Après un push pour corriger un commit distant
   - ☐ Avant un push pour modifier le dernier commit
   - ☐ Pour supprimer une branche
   - ☐ Pour fusionner deux commits
2. Pour annuler un commit déjà pushé proprement, on utilise :
   - ☐ `git reset HEAD~1`
   - ☐ `git revert HEAD`
   - ☐ `git push --force`
   - ☐ `git branch -d`
3. `git stash` permet de :
   - ☐ Supprimer des fichiers définitivement
   - ☐ Mettre de côté des modifications non commitées
   - ☐ Créer une branche temporaire
   - ☐ Annuler le dernier commit
4. Un `git push` est refusé (`rejected`), je fais en premier :
   - ☐ `git push --force`
   - ☐ `git reset HEAD~1`
   - ☐ `git pull origin main`
   - ☐ `git branch -d main`
5. `git reset HEAD~1` :
   - ☐ Supprime définitivement les modifications
   - ☐ Annule le dernier commit en conservant les modifs
   - ☐ Crée une nouvelle branche
   - ☐ Fusionne deux branches
6. Quand je suis perdu dans Git, je fais en premier :
   - ☐ `git log --all`
   - ☐ `git reset`
   - ☐ `git status`
   - ☐ `git stash`

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Réponses : Erreurs courantes

1. `git commit --amend` s'utilise :
   - ❌ ~~Après un push pour corriger un commit distant~~
   - ✅ Avant un push pour modifier le dernier commit
   - ❌ ~~Pour supprimer une branche~~
   - ❌ ~~Pour fusionner deux commits~~
2. Pour annuler un commit déjà pushé proprement, on utilise :
   - ❌ ~~`git reset HEAD~1`~~
   - ✅ `git revert HEAD`
   - ❌ ~~`git push --force`~~
   - ❌ ~~`git branch -d`~~
3. `git stash` permet de :
   - ❌ ~~Supprimer des fichiers définitivement~~
   - ✅ Mettre de côté des modifications non commitées
   - ❌ ~~Créer une branche temporaire~~
   - ❌ ~~Annuler le dernier commit~~
4. Un `git push` est refusé (`rejected`), je fais en premier :
   - ❌ ~~`git push --force`~~
   - ❌ ~~`git reset HEAD~1`~~
   - ✅ `git pull origin main`
   - ❌ ~~`git branch -d main`~~
5. `git reset HEAD~1` :
   - ❌ ~~Supprime définitivement les modifications~~
   - ✅ Annule le dernier commit en conservant les modifs
   - ❌ ~~Crée une nouvelle branche~~
   - ❌ ~~Fusionne deux branches~~
6. Quand je suis perdu dans Git, je fais en premier :
   - ❌ ~~`git log --all`~~
   - ❌ ~~`git reset`~~
   - ✅ `git status`
   - ❌ ~~`git stash`~~

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Questions : Collaboration & flux Git

1. Les quatre zones de Git sont :
   - ☐ Working Dir, Staging, Repo local, Remote
   - ☐ Local, Distant, Branche, Index
   - ☐ Commit, Push, Pull, Merge
   - ☐ HEAD, main, origin, fetch
2. `git fetch` se distingue de `git pull` car il :
   - ☐ Crée une nouvelle branche automatiquement
   - [ ] Télécharge les nouveautés sans modifier ton travail
   - ☐ Fusionne automatiquement les branches
   - ☐ Supprime les branches distantes obsolètes
3. `origin` dans `git push origin main` est :
   - ☐ Un mot-clé réservé de Git
   - ☐ Le nom de la branche principale
   - ☐ Un alias pointant vers l'URL du dépôt distant
   - ☐ Une commande de configuration
4. En collaboration, chaque développeur devrait travailler :
   - ☐ Directement sur `main`
   - ☐ Sur sa propre branche dédiée
   - ☐ Sur une copie locale sans jamais pusher
   - ☐ Sur la branche du collègue
5. `git pull` est équivalent à :
   - ☐ `git fetch` uniquement
   - ☐ `git merge` uniquement
   - ☐ `git fetch` + `git merge`
   - ☐ `git clone` + `git push`
6. La convention Conventional Commits utilise les préfixes :
   - ☐ `save:` , `undo:` , `sync:`
   - ☐ `feat:` , `fix:` , `docs:`
   - ☐ `push:` , `pull:` , `merge:`
   - ☐ `v1:` , `v2:` , `v3:`

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Réponses : Collaboration & flux Git

1. Les quatre zones de Git sont :
   - ✅ Working Dir, Staging, Repo local, Remote
   - ❌ ~~Local, Distant, Branche, Index~~
   - ❌ ~~Commit, Push, Pull, Merge~~
   - ❌ ~~HEAD, main, origin, fetch~~
2. `git fetch` se distingue de `git pull` car il :
   - ❌ ~~Crée une nouvelle branche automatiquement~~
   - ✅ Télécharge les nouveautés sans modifier ton travail
   - ❌ ~~Fusionne automatiquement les branches~~
   - ❌ ~~Supprime les branches distantes obsolètes~~
3. `origin` dans `git push origin main` est :
   - ❌ ~~Un mot-clé réservé de Git~~
   - ❌ ~~Le nom de la branche principale~~
   - ✅ Un alias pointant vers l'URL du dépôt distant
   - ❌ ~~Une commande de configuration~~
4. En collaboration, chaque développeur devrait travailler :
   - ❌ ~~Directement sur `main`~~
   - ✅ Sur sa propre branche dédiée
   - ❌ ~~Sur une copie locale sans jamais pusher~~
   - ❌ ~~Sur la branche du collègue~~
5. `git pull` est équivalent à :
   - ❌ ~~`git fetch` uniquement~~
   - ❌ ~~`git merge` uniquement~~
   - ✅ `git fetch` + `git merge`
   - ❌ ~~`git clone` + `git push`~~
6. La convention Conventional Commits utilise les préfixes :
   - ❌ ~~`save:`, `undo:`, `sync:`~~
   - ✅ `feat:`, `fix:`, `docs:`
   - ❌ ~~`push:`, `pull:`, `merge:`~~
   - ❌ ~~`v1:`, `v2:`, `v3:`~~

Récapitulatif des réponses (1/3)

Générales

1. Git est principalement :
   - ✅ Un gestionnaire de versions distribué
2. GitHub est :
   - ✅ Une plateforme web pour héberger du code Git
3. Qui a créé Git ?
   - ✅ Linus Torvalds
4. Un dépôt Git contient :
   - ✅ L'historique des modifications
5. Git permet de :
   - ✅ Travailler à plusieurs en toute sécurité
6. L'historique Git est :
   - ✅ Un graphe orienté acyclique (DAG)

Concepts de base

1. Que signifie "committer" ?
   - ✅ Enregistrer un ensemble de modifications
2. Une branche est :
   - ✅ Une copie de l'historique principal pour travail
3. L'index (staging area) est :
   - ✅ Une zone tampon avant commit
4. HEAD désigne :
   - ✅ Le dernier commit pointé
5. Un commit contient :
   - ✅ Un identifiant unique
6. Une "timeline" Git est :
   - ✅ Une chronologie des commits

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Récapitulatif des réponses (2/3)

Commandes Git

1. `git init` :
   - ✅ Crée un dépôt local
2. `git add` :
   - ✅ Ajoute des fichiers au staging
3. `git commit -m` :
   - ✅ Sauvegarde les modifications ajoutées
4. `git clone` :
   - ✅ Copie un dépôt distant localement
5. `git push origin main` :
   - ✅ Envoie les commits locaux sur le dépôt distant
6. `git merge` :
   - ✅ Fusionne deux branches

Bonnes pratiques

1. Un bon message de commit doit :
   - ✅ Être court et clair
2. Un commit atomique signifie :
   - ✅ Qu'il traite une seule chose à la fois
3. `.gitignore` permet :
   - ✅ D'exclure certains fichiers du suivi
4. Il est recommandé de committer :
   - ✅ Après chaque modification logique
5. Nommer une branche `feature/` indique :
   - ✅ Une branche dédiée à une nouvelle fonctionnalité
6. Un commit fourre-tout :
   - ✅ Peut rendre l'historique confus

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Récapitulatif des réponses (3/3)

Erreurs courantes

1. `git commit --amend` s'utilise :
   - ✅ Avant un push pour modifier le dernier commit
2. Pour annuler un commit déjà pushé proprement :
   - ✅ `git revert HEAD`
3. `git stash` permet de :
   - ✅ Mettre de côté des modifications non commitées
4. Push refusé, que faire en premier :
   - ✅ `git pull origin main`
5. `git reset HEAD~1` :
   - ✅ Annule le dernier commit en conservant les modifs
6. Première commande quand on est perdu :
   - ✅ `git status`

Collaboration & flux Git

1. Les quatre zones de Git :
   - ✅ Working Dir, Staging, Repo local, Remote
2. `git fetch` se distingue de `git pull` car il :
   - ✅ Télécharge les nouveautés sans modifier ton travail
3. `origin` est :
   - ✅ Un alias pointant vers l'URL du dépôt distant
4. En collaboration, chaque développeur devrait travailler :
   - ✅ Sur sa propre branche dédiée
5. `git pull` est équivalent à :
   - ✅ `git fetch` + `git merge`
6. Conventional Commits utilise les préfixes :
   - ✅ `feat:`, `fix:`, `docs:`

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

🎯 Conclusion du quiz

Félicitations d'être allé au bout de ce quiz complet ! ✅

Tu viens de valider :

- Tes connaissances générales sur Git 🧠
- La maîtrise de ses concepts de base ⚙️
- La compréhension des commandes essentielles 💻
- L'adoption de bonnes pratiques pour un historique clair 👌
- La gestion des erreurs courantes 🛠️
- Les bases de la collaboration en équipe 🤝

👉 Si tu as fait des erreurs, relis les chapitres correspondants et n'hésite pas à refaire les manipulations.

> 🧔 Git devient vite naturel avec de la pratique. Continue à explorer, à versionner et à collaborer 🚀



Conclusion du cours

🚀 Bravo, mission accomplie !

Voici le terme du parcours "**Git pour débutants**" 👏

Tu sais maintenant :

- 🧭 Ce qu'est Git et pourquoi il est essentiel aujourd'hui
- ⚙️ Manipuler les notions de dépôt, commit, branche, index et historique
- 🌐 Utiliser Git en local, en ligne de commande, et le connecter à GitHub ou GitLab
- 🤝 Gérer des projets collaboratifs en toute sécurité
- 📋 Respecter de bonnes pratiques pour un historique clair

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

🌱 Et maintenant ?

Git, c'est un outil **qui se maîtrise par la pratique**.

Voici quelques pistes pour progresser :

- 📚 Continue à versionner tes projets personnels, même simples
- 🧪 Expérimente avec les branches, conflits, et fusion dans des environnements tests
- 👀 Observe les bonnes pratiques sur des projets open source
- 🤓 Explore des commandes avancées comme `rebase`, `tag`, `log --graph`

Tu as les bases solides pour devenir un contributeur Git efficace !

🔗 Liens utiles et ressources

Voici quelques liens recommandés pour approfondir :

- 🌐 Documentation officielle de Git
- 📖 Article Wikipédia sur Git
- 🎓 Try Git (tutoriel interactif)
- 🐙 GitHub Docs
- 🦊 GitLab Docs
- 🔍 Visual Git Guide (git-school)
- 📘 Pro Git Book (gratuit, complet, par Scott Chacon)

N’hésite pas à les consulter et à les bookmarker !

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Une dernière chose...

N’oublie jamais :

> "Un bon commit est comme une phrase bien écrite : il dit ce qu’il fait, et il le fait bien." ✍️

À toi de jouer, et bonne route sur tes projets ! 🚀

> 🧔 Et comme le dit la chanson : `🎶 On va commit! 🎶`

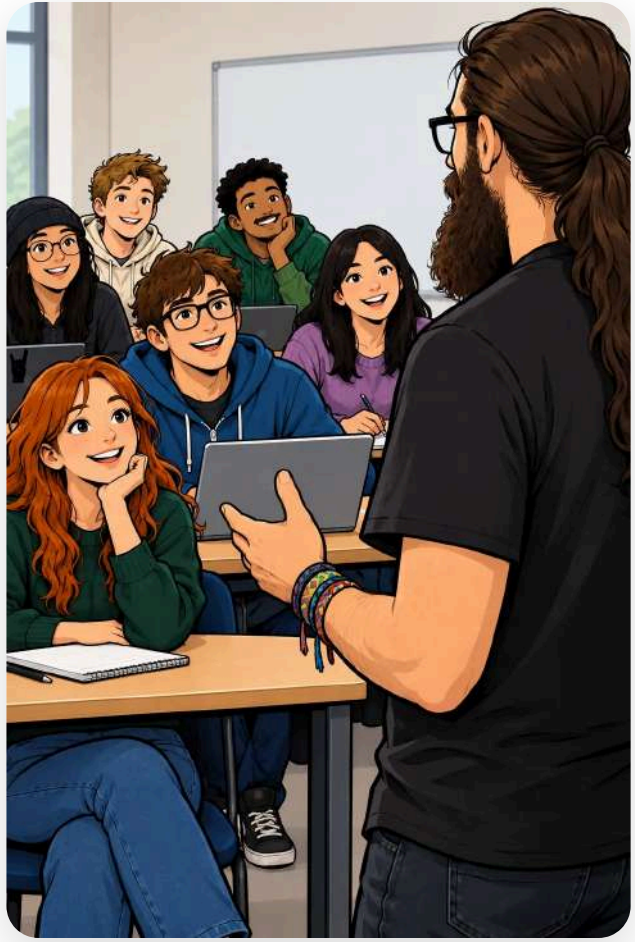
Questions

Conclusion du cours

Vos questions ✋

Vos demandes d'ajout ✋

- ?



🤝 Merci de ton attention ! 🤝

Reste curieux 🧐 et amuse-toi ! 🎮 🎉

Ce cours t'a été présenté par

 Moi-même, Nathaniel Vaur Henel 🤘

 mon site de partage autour du dév : dev-sensei.digicrafters.fr

 mon Framagit perso, GitLab éthique, libre & français : framagit.org/nathvh/me

 mon GitHub perso, mais inactif : github.com/Nathaniel-Vaur-Henel

 mon LinkedIn, pour suivre mon parcours pro : linkedin.com/in/nathaniel-vaur-henel

 mon Discord pro, pour échanger ou me poser des questions : `@nathanielvaurhenel`